

# Heaps and Priority Queues

DSA27 — Lecture 12 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 12 · Fall 2026

## Where We Are

---

# A queue that does not wait its turn

Week 7's queue is **fair**: first in, first out.

An emergency department is not fair, on purpose:

- a patient with chest pain is seen **before** one with a sprained ankle
- who arrived first matters only between equally urgent patients

We need a queue that serves the **most urgent** item: a **priority queue**.

1. The priority queue ADT, and three ways to build it badly
2. The heap: a **shape** and an **order**
3. The array **is** the tree:  $2i + 1$ ,  $2i + 2$ ,  $(i - 1) // 2$
4. Sift up (push) and sift down (pop):  $O(\log n)$
5. Build-heap in  $O(n)$  — the summation, and a measurement
6. The priority queue on top; top-k; heap sort revisited

# The Priority Queue ADT

---

# Serve the most urgent

| Operation                            | Meaning                                                           |
|--------------------------------------|-------------------------------------------------------------------|
| <code>enqueue(item, priority)</code> | add an item with a priority                                       |
| <code>dequeue()</code>               | remove and return the item with the <b>lowest</b> priority number |
| <code>peek()</code>                  | look at that item without removing it                             |
| <code>is_empty(), len(pq)</code>     |                                                                   |

“Priority 1” means “most urgent” — as in triage, and in `dsa/heap.py`.

## Three obvious designs, and what each costs

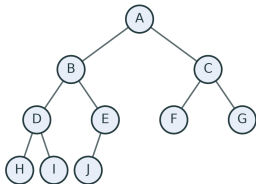
| Storage            | enqueue     | dequeue     | Why                                                |
|--------------------|-------------|-------------|----------------------------------------------------|
| unsorted array     | $O(1)$      | $O(n)$      | append; then <b>search</b> for the minimum         |
| sorted array       | $O(n)$      | $O(1)$      | <b>shift</b> to insert in order; take from the end |
| sorted linked list | $O(n)$      | $O(1)$      | <b>walk</b> to the insertion point                 |
| <b>binary heap</b> | $O(\log n)$ | $O(\log n)$ | today                                              |

# The Heap

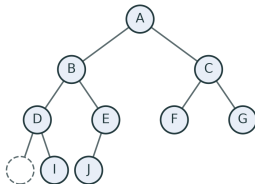
---

# Shape: a complete binary tree

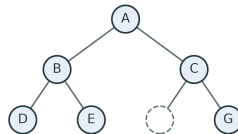
A heap's shape: a complete binary tree — so the array has no holes



complete: every level full,  
the last one filled from the left



not complete: a hole in the  
last level (index 7)

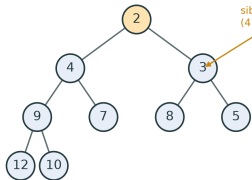


not complete: a missing node  
(index 5) before the last

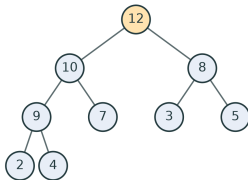
**Complete:** every level is full, except perhaps the last, which is filled **from the left**, with no gaps.

# Order: the heap property

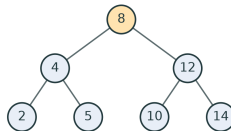
The heap property: each parent beats its children — nothing more



min-heap: every parent  $\leq$  its children



max-heap: every parent  $\geq$  its children



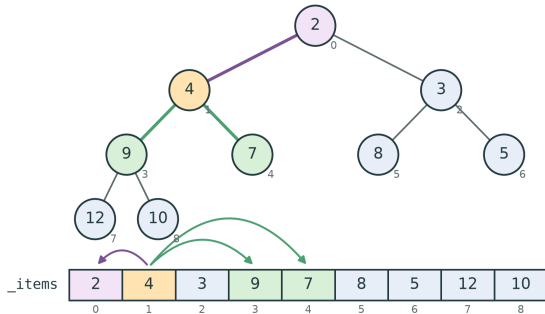
a BST (Week 11): left < parent < right  
— a stronger, different order

**Min-heap:** every parent  $\leq$  its children. **Max-heap:** every parent  $\geq$  its children.

So the best item is at the **root**. Siblings are **not** ordered.

# The array is the tree

The array is the tree: no nodes, no pointers — the shape is arithmetic



$i = 1$  (value 4)

parent  $(i - 1) // 2 = 0$

left  $2i + 1 = 3$

right  $2i + 2 = 4$

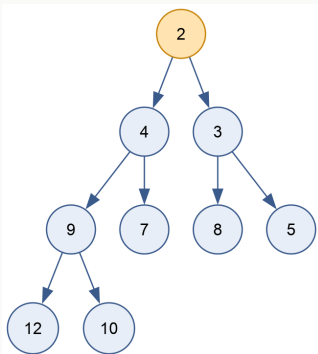
level  $k$  holds indices

$2^k - 1 \dots 2^{k+1} - 2$

Node  $i$ : children at  $2*i + 1$  and  $2*i + 2$ ; parent at  $(i - 1) // 2$ .

## See it: draw\_array\_as\_tree

```
from viz.draw import draw_array_as_tree  
draw_array_as_tree([2, 4, 3, 9, 7, 8, 5, 12, 10], highlight=0)
```



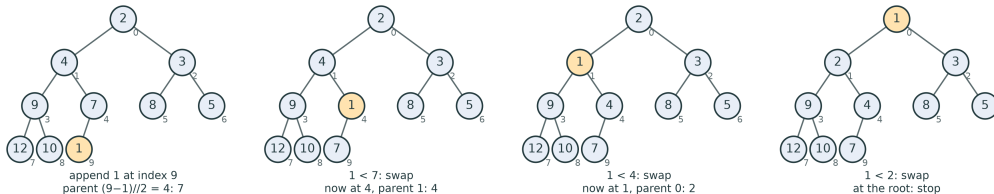
A plain list, drawn as the tree it already is. No conversion.

**Sift Up: push**

---

# Append, then sift up

push(1): append at the end, then sift up — 3 swaps, one per level



1. **Append** the new value at the end — the shape stays complete.
2. While it beats its **parent**, swap it with the parent.

## Trace: push(1)

| Step   | 1 at | parent index       | parent value | Decision                        |
|--------|------|--------------------|--------------|---------------------------------|
| append | 9    | $(9 - 1) // 2 = 4$ | 7            | $1 < 7 \rightarrow \text{swap}$ |
|        | 4    | $(4 - 1) // 2 = 1$ | 4            | $1 < 4 \rightarrow \text{swap}$ |
|        | 1    | $(1 - 1) // 2 = 0$ | 2            | $1 < 2 \rightarrow \text{swap}$ |
|        | 0    | —                  | —            | root: stop                      |

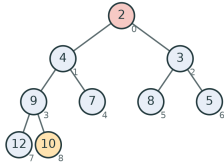
Result: [1, 2, 3, 9, 4, 8, 5, 12, 10, 7]. Three swaps: one per level.

**Sift Down: pop**

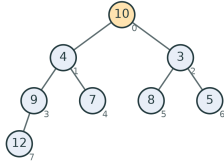
---

# Remove the root, fill the hole from the end

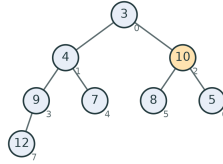
**pop():** move the last leaf to the root, then sift down past the better child



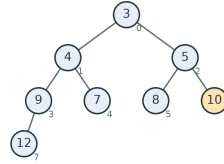
pop() returns the root, 2;  
the last leaf, 10, will fill the hole



10 at the root; children 4, 3  
better child: 3 → swap



10 at 2; children 8, 5  
better child: 5 → swap

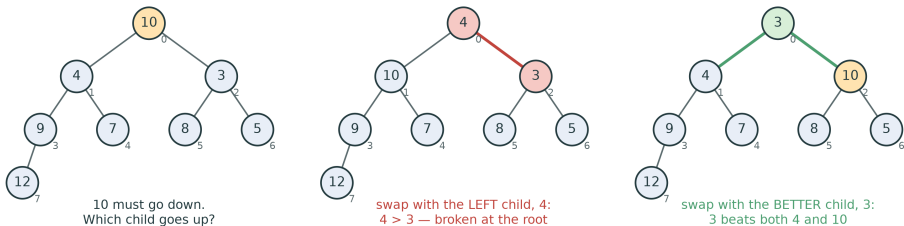


10 at 6: no children  
stop — valid again

1. Keep the root — it is the answer.
2. Move the **last** item into the root; shrink the array by one.
3. While it is beaten by its **better child**, swap with that child.

# The trap: swap with the better child

The sift-down trap: the child that moves up becomes the parent of its sibling



The child that moves up becomes the **parent of its sibling** — so it must beat the sibling too.

## Both sifts are $O(\log n)$

| Operation                   | Walks        | Comparisons per level | Worst case                               |
|-----------------------------|--------------|-----------------------|------------------------------------------|
| push $\rightarrow$ sift up  | leaf to root | 1                     | $\lfloor \log_2 n \rfloor$ swaps         |
| pop $\rightarrow$ sift down | root to leaf | 2                     | $2 \lfloor \log_2 n \rfloor$ comparisons |
| peek                        | —            | —                     | $O(1)$                                   |

A heap of a million items: at most **20** levels.

**Build-Heap in  $O(n)$**

---

## Two ways to make a heap from $n$ values

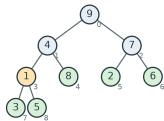
**Way 1 —  $n$  pushes.** Start empty; push each value.  $n \times O(\log n) = O(n \log n)$ .

**Way 2 — heapify, bottom up.** Leave the values where they are. Sift **down** every parent, from the **last** parent back to the root.

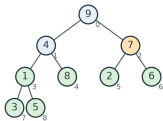
Way 2 is  $O(n)$ .

# Trace: `heapify([9, 4, 7, 1, 8, 2, 6, 3, 5])`

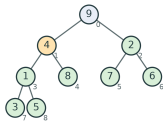
`heapify([9, 4, 7, 1, 8, 2, 6, 3, 5])`: sift down from the last parent, index 3, back to the root



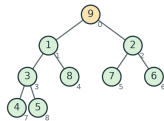
`sift_down(3)`: 1 stays



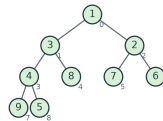
`sift_down(2)`: 7 moves down 1 level



`sift_down(1)`: 4 moves down 2 levels



`sift_down(0)`: 9 moves down 3 levels



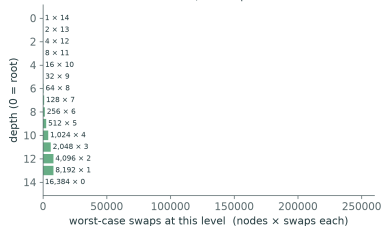
done: a valid min-heap  
14 comparisons in all

$n = 9$ , last parent  $= 9 // 2 - 1 = \mathbf{3}$ . Sift down 3, 2, 1, 0.

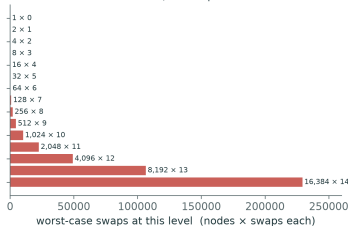
# Why heapify is $O(n)$ : count the work level by level

A perfect heap of  $n = 32,767$ : most nodes are near the bottom — so let them do the least work

heapify: sift DOWN — at most (height below) swaps  
total at most 32,752 swaps = 1.00  $n$



$n$  pushes: sift UP — at most (depth) swaps  
total at most 425,986 swaps = 13.00  $n$

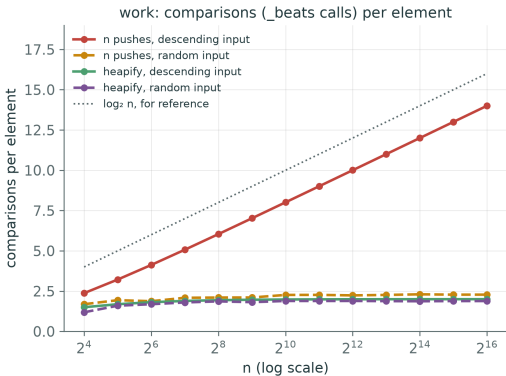
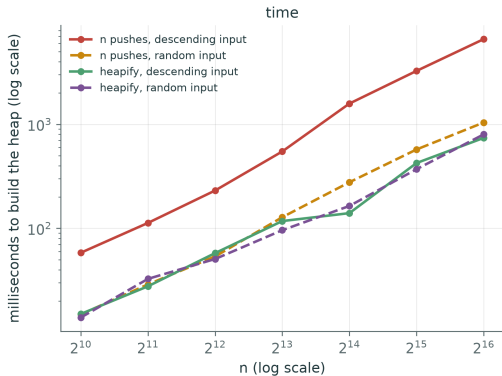


About  $n/2^{h+1}$  nodes sit at **height  $h$** ; each sifts down at most  $h$  levels:

$$\sum_{h \geq 0} \frac{n}{2^{h+1}} \cdot h = \frac{n}{2} \sum_{h \geq 0} \frac{h}{2^h} = \frac{n}{2} \cdot 2 = n$$

# Measured: heapify against n pushes

Measured: heapify against n pushes



## The Priority Queue on Top

---

## PriorityQueue: a MinHeap of triples

`enqueue(item, priority)` pushes the triple

`(priority, counter, item)`

and adds 1 to counter. `dequeue()` pops a triple and returns its **item**.

- Python compares tuples **left to right**: priority first.
- counter is unique, so the comparison **never reaches** item.

## Trace: triage

| Operation    | Heap array: (priority, counter, item) | Returns |
|--------------|---------------------------------------|---------|
| enqueue A, 2 | (2,0,A)                               |         |
| enqueue C, 1 | (1,1,C) (2,0,A)                       |         |
| enqueue K, 3 | (1,1,C) (2,0,A) (3,2,K)               |         |
| enqueue L, 1 | (1,1,C) (1,3,L) (3,2,K) (2,0,A)       |         |
| dequeue      | (1,3,L) (2,0,A) (3,2,K)               | C       |
| dequeue      | (2,0,A) (3,2,K)                       | L       |

A broken arm · C chest pain · K sprained ankle · L allergic reaction

## Where priority queues are used

- **Scheduling:** an operating system runs the highest-priority ready process; a print server, a build server, a job queue.
- **Triage:** emergency departments; bug trackers (“P1 before P3”).
- **Event simulation:** the next event is the one with the smallest time.
- **Top-k:** the  $k$  largest of  $n$  values, in  $O(n \log k)$ .
- **Graph algorithms:** Dijkstra’s shortest paths, Prim’s spanning tree, Huffman coding — *beyond this course’s bylaw*.

## Top-k: keep a small heap, not a big one

The k **largest** of n values:

1. Keep a **min**-heap of at most k values.
2. For each value: if the heap has fewer than k, push it. Otherwise, if the value beats the heap's **smallest** (its root), pop the root and push the value.
3. The heap now holds the k largest.

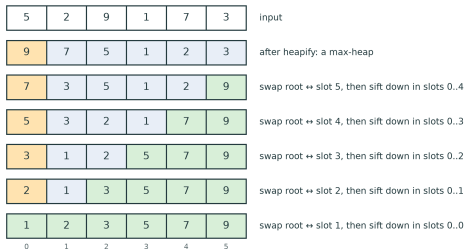
$O(n \log k)$  time,  $O(k)$  space. Sorting everything:  $O(n \log n)$  and  $O(n)$ .

## Heap Sort, Revisited

---

# Week 10 was a heap in disguise

**heap\_sort (Week 10): heapify a max-heap, then pop the maximum to the back,  $n - 1$  times**



1. heapify the array as a **max**-heap:  $O(n)$ .
2. Repeat  $n - 1$  times: swap the root (the maximum) with the **last** item of the heap; the heap shrinks by one; sift the new root down:  $O(\log n)$ .

$O(n \log n)$  always,  $O(1)$  extra space, **not stable**.

## **This Week**

---

## Exercises: dsa/heap.py

| Method                                | Target               | The trap                                                                     |
|---------------------------------------|----------------------|------------------------------------------------------------------------------|
| <code>_sift_up</code>                 | $O(\log n)$          | parent is <code>(i - 1) // 2</code> ;<br>stop at the root                    |
| <code>_sift_down</code>               | $O(\log n)$          | the <b>better</b> child; a missing<br>right child                            |
| <code>push</code> , <code>peek</code> | $O(\log n)$ , $O(1)$ | <code>IndexError</code> on an empty<br><code>peek</code>                     |
| <code>pop</code>                      | $O(\log n)$          | last item to the root; the<br>one-item heap                                  |
| <code>heapify</code>                  | $O(n)$               | from <code>n // 2 - 1</code> <b>down</b><br>to 0 — not <code>n</code> pushes |
| <code>is_valid</code>                 | $O(n)$               | check <b>every</b> child against<br>its parent                               |
| <code>PriorityQueue</code>            | $O(\log n)$          | the triple; return the <b>item</b>                                           |

```
pytest tests/test_heap.py -v
```

## Homework 12 — before Lecture 13

1. **Implement** `dsa/heap.py` until all 28 tests pass.
2. **Trace** by hand, as trees and as arrays: push 6, 2, 8, 1, 5, 3 into an empty `MinHeap`; then pop twice.
3. **Heapify** `[3, 9, 2, 1, 4, 5, 8, 7, 6, 0]` by hand. Count the swaps.
4. **Measure** heapify against  $n$  pushes on descending input in `notebooks/12-heaps.ipynb`, and count `_beats` calls with a subclass.

## Summary

---

## Seven things to keep

1. A priority queue serves the **most urgent** item; a heap gives  $O(\log n)$  enqueue **and** dequeue.
2. **Shape:** a complete binary tree — height  $\lfloor \log_2 n \rfloor$ .
3. **Order:** every parent beats its children; siblings are unordered.
4. The array **is** the tree: children  $2i + 1, 2i + 2$ ; parent  $(i - 1) // 2$ .
5. push = append + **sift up**; pop = last to root + **sift down** past the **better** child.
6. heapify sifts down from  $n // 2 - 1$  to 0:  $O(n)$ , because most nodes are near the bottom.
7. PriorityQueue pushes (priority, counter, item): the counter breaks ties — first come, first served.

**Week 13 — Hash tables.** From  $O(\log n)$  to  $O(1)$  — **on average**. What “average” promises, and what breaks it.